

- 1 Autoencoder Fundamentals
- 2 Types of Autoencoders
- 3 References

3 References

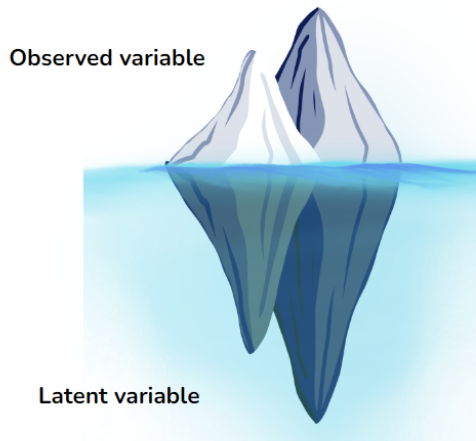
Unlabeled Data Usage in Deep Learning



How can we use unlabeled data in our deep neural networks?

Image: Source

What Is Latent Variable?



Latent Variables in Deep Learning

We can train **deep neural networks** using **unlabeled data** to infer **latent variables** in order to:

- Compress data
- Generate new data
- Denoise Data
- Using latent variables as features for various tasks like classification
- ...

Autoencoder: Background

Using latent variables as a foundation, an unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data.

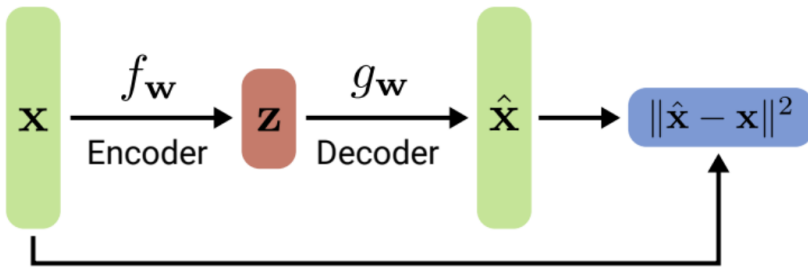


Why do we care about a low-dimensional z ?

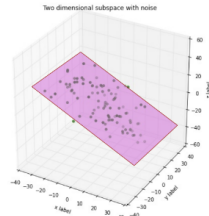
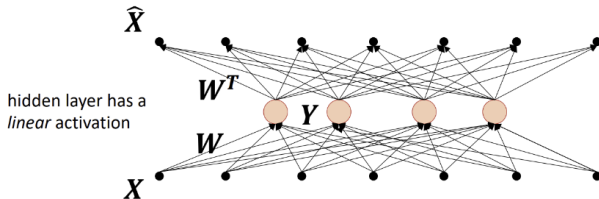
"Encoder" learns mapping from the data, x , to a low-dimensional latent space, z

Autoencoders

- ① These models map between **observation space** $\mathbf{x} \in \mathbb{R}^D$ and **latent space** $\mathbf{z} \in \mathbb{R}^Q$:
 - **Encoder** $f_w: \mathbf{x} \mapsto \mathbf{z}$ The "Analysis" net which computes the hidden representation
 - **Decoder** $g_w: \mathbf{z} \mapsto \mathbf{x}$ The "Synthesis" which recomposes the data from the hidden representation
- ② Models are linear or non-linear, deterministic or stochastic, with/without encoder.



Autoencoder Without a Non-linearity



$$\mathbf{Y} = \mathbf{W}\mathbf{X}$$

$$\hat{\mathbf{X}} = \mathbf{W}^T \mathbf{Y}$$

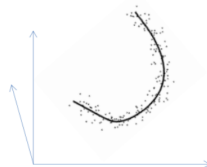
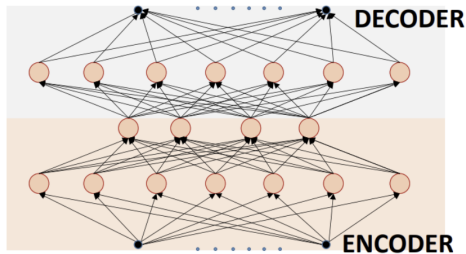
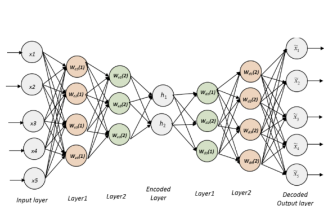
$$E = \|\mathbf{X} - W^T W \mathbf{X}\|^2$$

Find W to minimize $\text{Avg}[E]$

- **This is similar to PCA**
 - The output of the hidden layer will be in the principal subspace
 - Even if the recomposition weights are different from the "analysis" weights

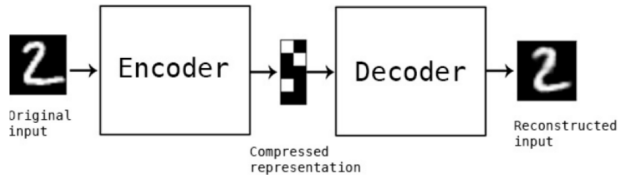
“Deep” AE

- **Deep nonlinear autoencoders** learn to project the data, not onto a subspace, but onto a nonlinear **manifold**.
- This manifold is the **image of the decoder**.
- This is a kind of **nonlinear dimensionality reduction**.



AE vs PCA

As we said, **PCA** can be described as

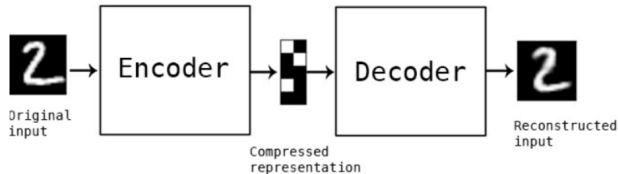


$$\min_W \sum \|\hat{x} - x\|^2$$

$$W^T W = I$$

$$\min_W \sum \|W^T Wx - x\|^2$$

And also **Autoencoders** can be thought of as a **non-linear PCA**.

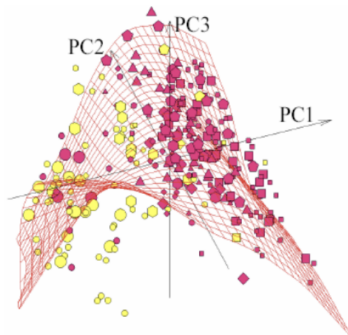
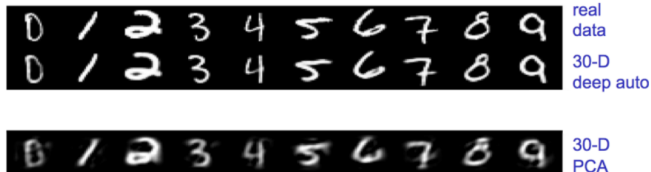


$$\min_{h,g} \sum \|\hat{x} - x\|^2$$

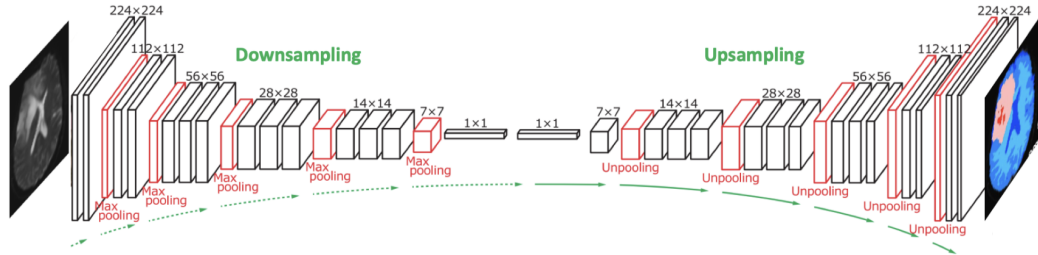
$$\min_{h,g} \sum \|g(f(x)) - x\|^2$$

AE vs PCA

Nonlinear autoencoders can learn more powerful codes for a given dimensionality, compared with linear autoencoders (PCA).



Fully Convolutional AE

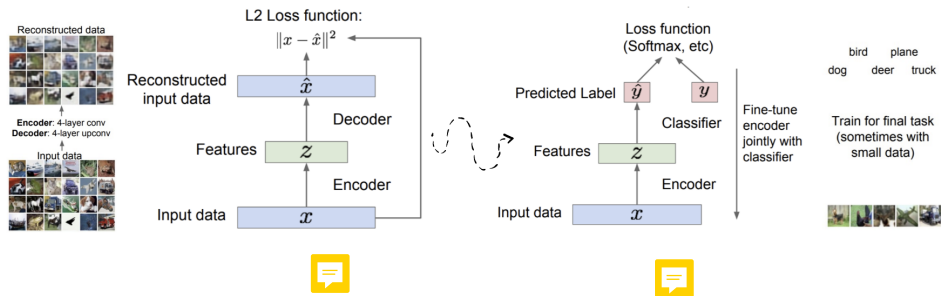


We will discuss more about "**Downsampling**" and "**Upsampling**" in the next section

Image from this link

Autoencoders As Pretrained Models

- **Feature Extraction:** Encoder **learns** data patterns and **compresses** them into useful features.
- **Supervised Model Initialization:** Use the **pretrained encoder** to initialize a supervised learning model.
- **Fine-Tuning:** Train the encoder and classifier **together** to improve performance.
- **Useful with Small Data:** Effective when training on **small datasets** by leveraging pretrained features.



AE Performance & Bottleneck Size (MNIST Example)

- **Reconstruction Error and Bottleneck Size:**

- A good autoencoder should minimize reconstruction error.
- The error depends heavily on the size of the z (the dimension of the latent space).

- **Experiment Setup:**

- CNN-based AE trained on MNIST with different bottleneck sizes:
 - (2, 1, 1), (4, 1, 1) & (32, 1, 1)

- 1 Autoencoder Fundamentals
- 2 Types of Autoencoders**
- 3 References

AE's: Sparse Autoencoders (Intuition)

Idea: Can we describe the input with a small set of “*attributes*”?
This might be a more *compressed* and *structured* representation.

Aside:

This idea originated in neuroscience, where researchers believe that the brain uses **sparse** representations (see “sparse coding”).

1. NOT structured - “dense”: most values non-zero



```
Pixel (0,0): #FE057D
Pixel (0,1): #FD0263
Pixel (0,2): #E1065F
```

Idea: “Sparse” representations are going to be more structured!

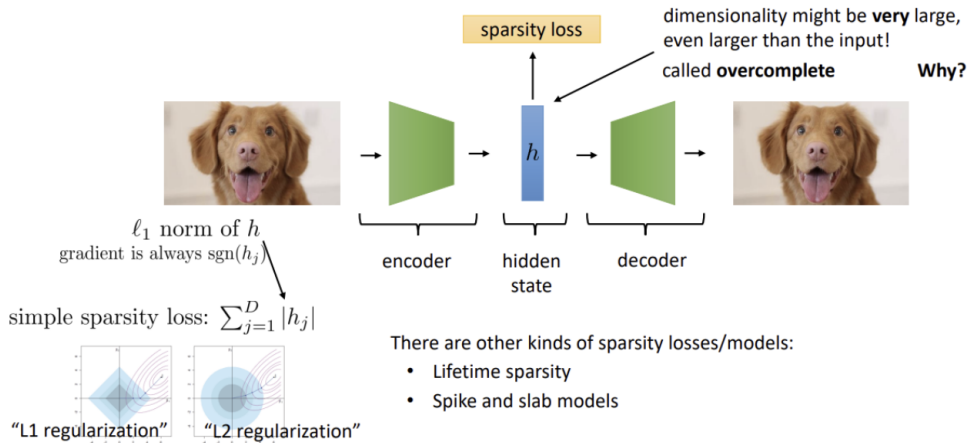
2. Very structured! - “sparse”: most values are zero



```
has_ears: 1
has_wings: 0
has_wheels: 0
```

There are many possible “attributes,” & most images don't have most of the attributes.

AE's: Sparse Autoencoder (Big Picture)



AE's: Sparse Autoencoders (Details)

- 1 Sparse Autoencoders try to minimize the following function.

$$L(x, g(f(x))) + \Omega(h)$$

- 2 The first term is loss for copying inputs.
- 3 The second term is sparsity penalty.
- 4 In general neural networks, we are trying to find the **maximum likelihood**: $p(x|\theta)$.
- 5 We often use $\log p(x|\theta)$ for simplification, from which we can get the loss function without regularization.
- 6 What about MAP (Maximum a posteriori)?

$$p(\theta|x) \propto p(x|\theta) \times p(\theta)$$

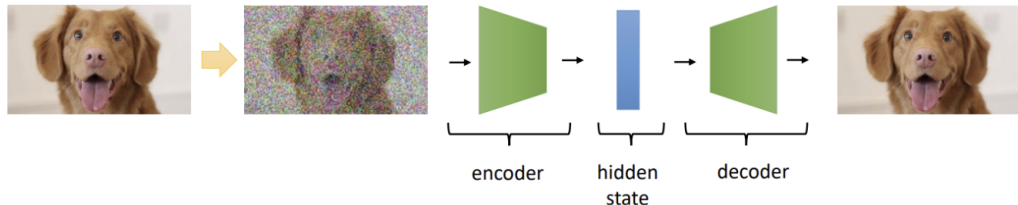
- 7 Maximizing the log of the above function yields:

$$\text{maximize } (\log p(x|\theta) + \log p(\theta))$$

- 8 The first term is the loss function, and the second term is the regularization penalty.

AE's: Denoising Autoencoders (Intuition)

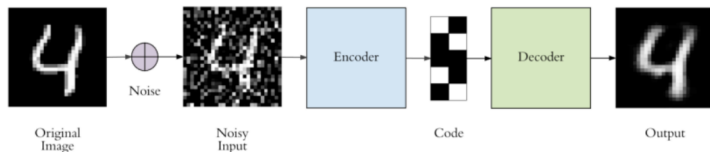
Idea: a good model that has learned meaningful structure should “fill in the blanks”



There are **many variants** on this basic idea, and this is one of the most widely used simple autoencoder designs.

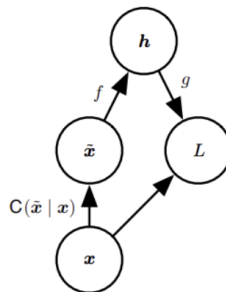
AE's: Denoising Autoencoders (Details)

- 1 **The denoising autoencoder (DAE)** is an autoencoder that receives a corrupted data point as input and is trained to predict the original, uncorrupted data point as its output.
- 2 **Traditional autoencoders minimize** $L(x, g(f(x)))$
 - L is a loss function penalizing $g(f(x))$ for being dissimilar from x , such as L_2 norm of difference: mean squared error.
- 3 **A DAE minimizes** $L(x, g(f(\tilde{x})))$
 - $\tilde{x}$ is a copy of x that is corrupted by some form of noise.
 - The autoencoder must undo this corruption rather than simply copying their input.



AE's: Denoising Autoencoders (Training Procedure)

① The DAE training procedure is

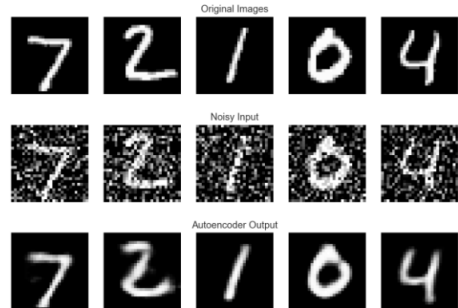
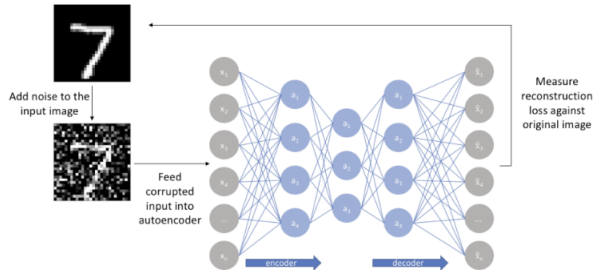


② We introduce a corruption process $C(\tilde{x}|x)$.

AE's: Denoising Autoencoders (Training Procedure)

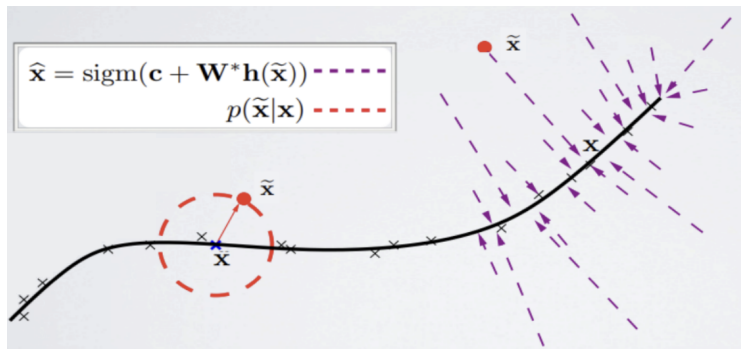
- 1 We introduce a corruption process $C(\tilde{x}|x)$.
- 2 The autoencoder then learns a reconstruction distribution $p_{\text{reconstruct}}(x|\tilde{x})$ estimated from training pairs $(x, \tilde{x})$ as follows:
 - Sample a training example x_i from the training data.
 - Sample a corrupted version $\tilde{x}_i$ from $C(\tilde{x}_i|x = x_i)$.
 - Use $(x, \tilde{x})$ as a training example for estimating the autoencoder reconstruction distribution $p_{\text{reconstruct}}(x|\tilde{x}) = p_{\text{decoder}}(x|h)$ with h the output of encoder $f(\tilde{x})$ and p_{decoder} typically defined by a decoder $g(h)$.
 - Typically we can simply perform gradient-based approximate minimization on the negative log-likelihood $-\log p_{\text{decoder}}(x|h)$.

AE's: Denoising Autoencoders (Result On MNIST)



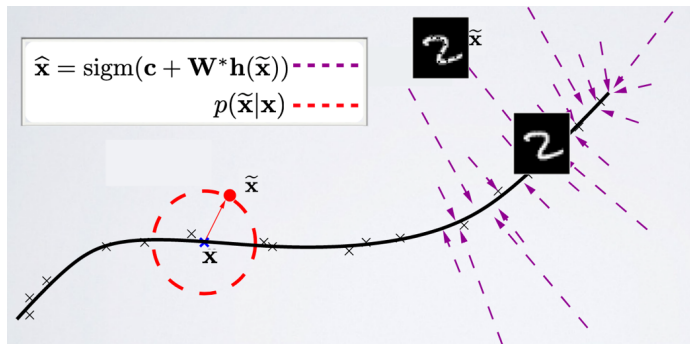
AE's: Denoising Autoencoders (It Isn't Lazy Network!)

- DAEs won't simply memorize the inputs and outputs (More robustness)
- Intuitively, a DAE learns a **projection** from a neighborhood of our training data back onto the training data



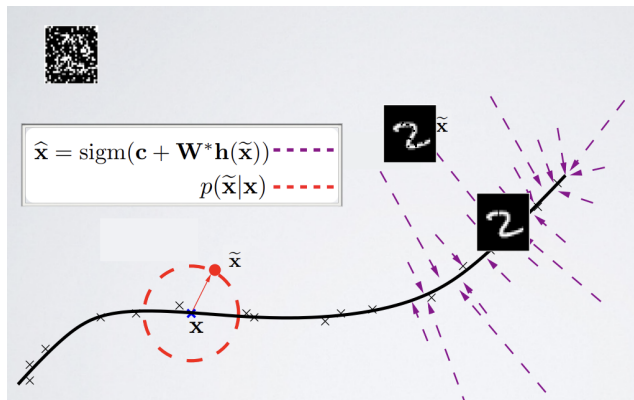
AE's: Denoising Autoencoders (It Isn't Lazy Network!)

- DAEs won't simply memorize the inputs and outputs (More robustness)
- Intuitively, a DAE learns a **projection** from a neighborhood of our training data back onto the training data



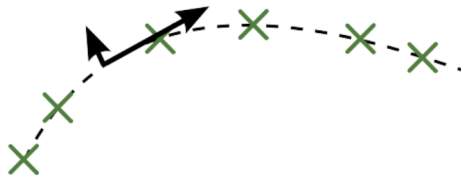
AE's: Denoising Autoencoders (It Isn't Lazy Network!)

- DAEs won't simply memorize the inputs and outputs (More robustness)
- Intuitively, a DAE learns a **projection** from a neighborhood of our training data back onto the training data

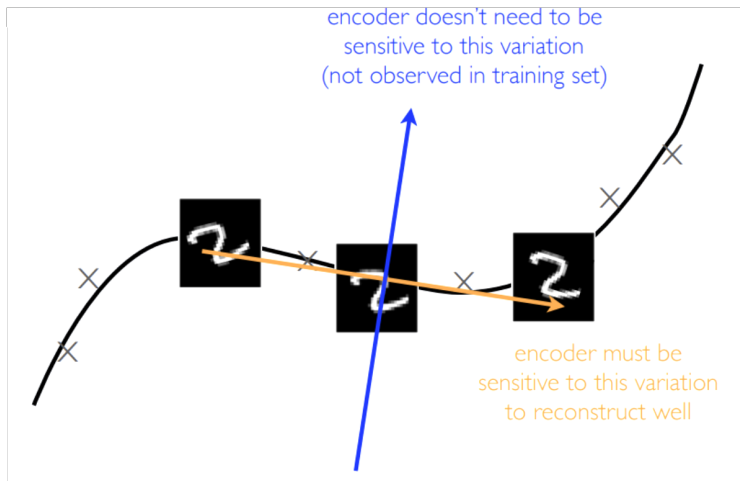


AE's: Contractive Autoencoder (Intuition)

- 1 Contractive autoencoders are explicitly encouraged to learn a manifold through their loss function (Alain and Bengio 2014).
- 2 **Desirable property:** Points close to each other in input space maintain that property in the latent space.
- 3 Method to avoid uninteresting solutions
- 4 Add an explicit term in the loss that penalizes that solution
- 5 We wish to extract features that only reflect variations observed in the training set
- 6 We would like to be invariant to other variations

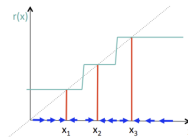


AE's: Contractive Autoencoder (Intuition)



AE's: Contractive Autoencoder (Details)

- ① Contractive autoencoder has an explicit regularizer on $h = f(x)$, encouraging the derivatives of f to be as small as possible.
- ② This will be true if $f(x) = h$ is continuous, has small derivatives.



- ③ We can use the [Frobenius Norm](#) of the [Jacobian Matrix](#) as a regularization term:

$$\Omega(f, x) = \lambda \left\| \frac{\partial f(x)}{\partial x} \right\|_F^2$$

- ④ These autoencoders are called *contractive* because they contract the neighborhood of input space into a smaller, localized group in latent space.
- ⑤ **Exercise:** What is the difference between DAE and CAE?

AE's: Contractive Autoencoder (Details)

- New loss function:

$$\underbrace{l\left(f\left(x^{(t)}\right)\right)}_{\text{autoencoder reconstruction}} + \lambda \underbrace{\left\|\nabla_{x^{(t)}} h\left(x^{(t)}\right)\right\|_F^2}_{\text{Jacobian of encoder}}$$

- where, for binary observations:

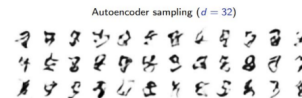
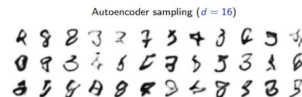
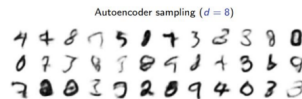
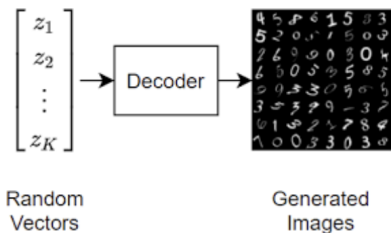
$$l\left(f\left(x^{(t)}\right)\right) = -\sum_k \left(x_k^{(t)} \log\left(\hat{x}_k^{(t)}\right) + \left(1 - x_k^{(t)}\right) \log\left(1 - \hat{x}_k^{(t)}\right)\right) \quad \left. \vphantom{l\left(f\left(x^{(t)}\right)\right)} \right\} \begin{array}{l} \text{encoders keeps} \\ \text{good information} \end{array}$$

$$\left\|\nabla_{x^{(t)}} h\left(x^{(t)}\right)\right\|_F^2 = \sum_j \sum_k \left(\frac{\partial h\left(x^{(t)}\right)_j}{\partial x_k^{(t)}}\right)^2 \quad \left. \vphantom{\left\|\nabla_{x^{(t)}} h\left(x^{(t)}\right)\right\|_F^2} \right\} \begin{array}{l} \text{encoder throws} \\ \text{away all information} \end{array}$$

→ encoders keep only good information

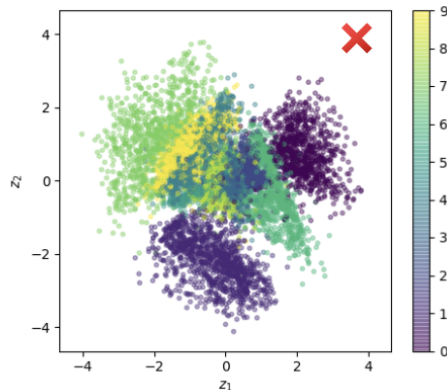
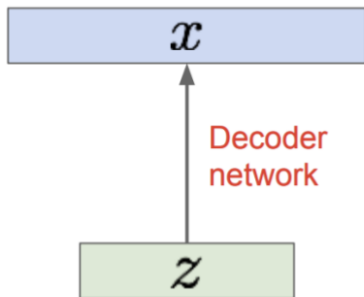
Can The Decoder Of AE Be Used For Data Generation?

- Autoencoders are effective data compressors but up to now do not yet show **data generation capabilities**.
- We already trained an AE on MNIST; then we use **random vectors** as input to the Decoder to generate new data.
 - Is the **size** of the bottleneck affecting the **quality** of the generated sample?



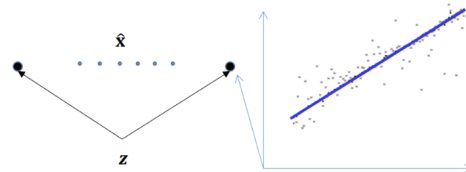
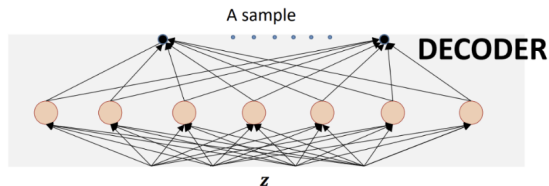
Can The Decoder Of AE Be Used For Data Generation?

Is **any random vector** we feed into decoder networks results in a valid output?



Can The Decoder Of AE Be Used For Data Generation?

- **Latent Variation:** Varying z generates outputs along a learned manifold.
- **Manifold Constraint:** Data lies within patterns learned from training.
- **Generation:** Outputs follow the training data distribution.



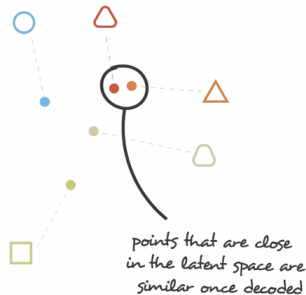
Latent Space Properties

- For the generation purpose, we need the latent space to show the following properties:
 - **Continuity:** Similar points in the latent space will be similar after decoding.
 - **Completeness:** Samples of the latent space lead to meaningful content after decoding.

Latent Space Properties: Continuity



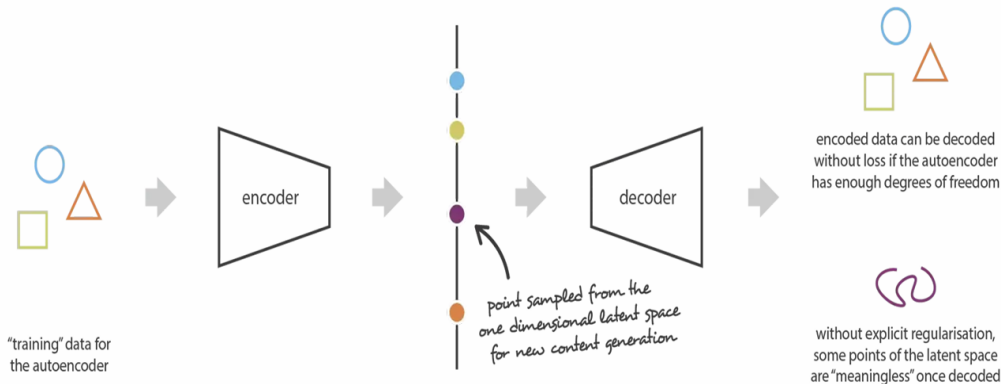
irregular latent space



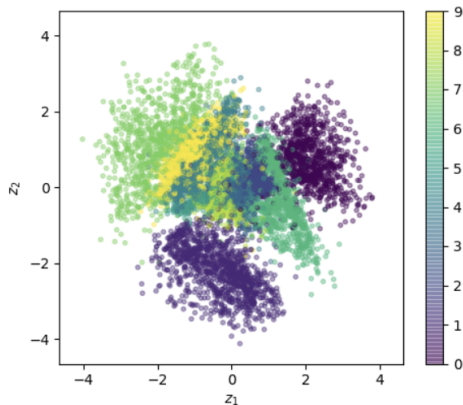
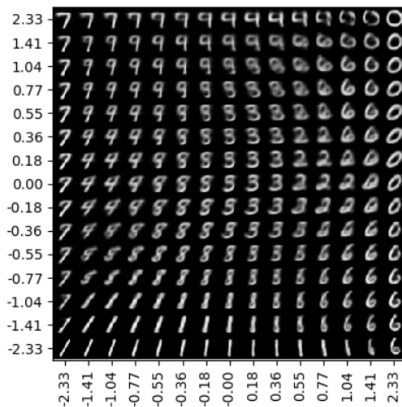
regular latent space



Latent Space Properties: Completeness

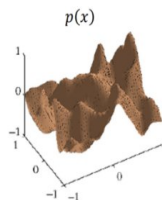
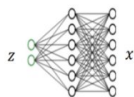


AE Latent Space - MNIST



What is the problem?

- $$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$



gib242 - 71 only (Sampling, Space, Estimates and Variational Autoencoders, 0-6h)

KL-Divergence

$D_{KL}(P||Q) = \sum_n P(n) \log \frac{P(n)}{Q(n)}$

$\xrightarrow{\text{continuous}}$

$D_{KL}(P||Q) = \int_n P(n) \log \frac{P(n)}{Q(n)} dn$

P: True Distribution

Q: Approximation

$D_{KL}(P||Q_1) \uparrow$

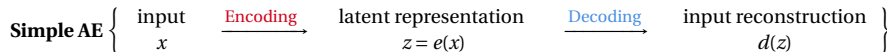
$D_{KL}(P||Q_2) \downarrow$

$D_{KL}(Q||P) \geq 0$

$D_{KL}(Q||P) = 0 \iff P = Q$

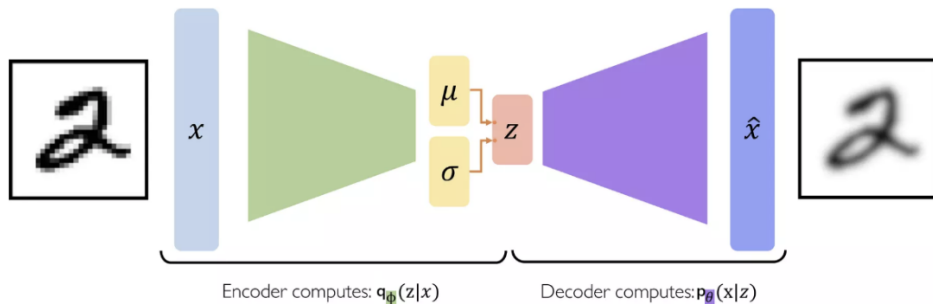
AE's: Variational Autoencoder (Intuition)

The key change that they added was that instead of having an encoder that maps points in X to points in Z , VAE's map points in X to distributions in Z . This allows a point in X to be indirectly mapped to several close neighbour points in Z .



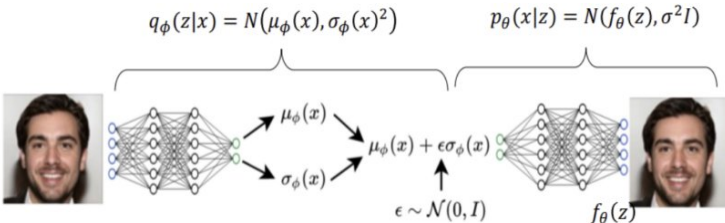
AE's: Variational Autoencoder (Big Picture)

The key change that they added was that instead of having an encoder that maps points in X to points in Z , VAE's map points in X to distributions in Z . This allows a point in X to be indirectly mapped to several close neighbour points in Z .



$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

AE's: Variational Autoencoder (Details)



VAE loss: $\mathcal{L}(x, \theta, \phi) =$

$$\underbrace{\|x^{(i)} - \hat{x}^{(i)}\|^2}_{\text{reconstruction loss}}$$

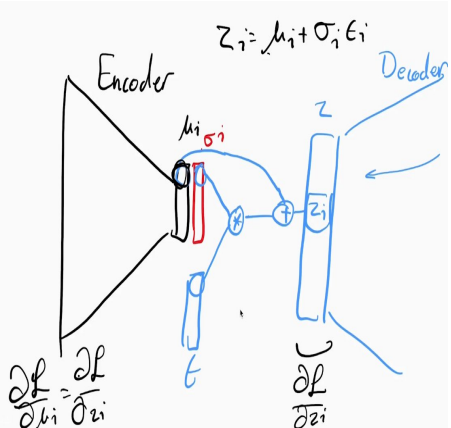
↓

$$f_{\theta}(z) = f_{\theta}(\mu_{\phi}(x^{(i)}) + \epsilon \sigma_{\phi}(x^{(i)}))$$

$$+ \underbrace{KL(q_{\phi}(z|x^{(i)}) \parallel \mathcal{N}(0, I))}_{\text{regularization term}}$$

↓

Common choice of prior on the latent distribution.



AE's: Variational Autoencoder (Details)

- Encourage encodings to be evenly distributed around the center of the latent space
- Penalize the network when it tries to memorize data

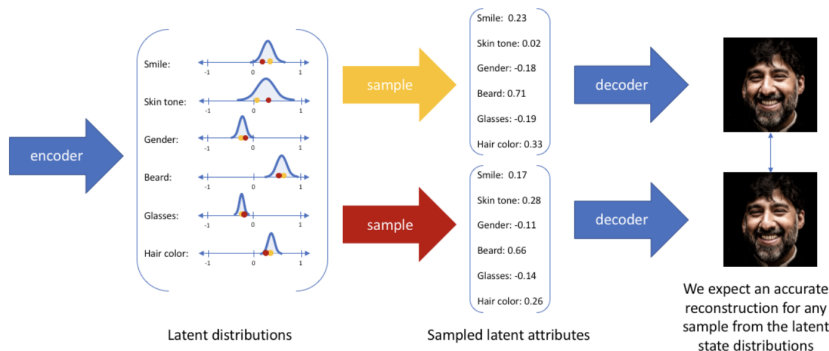
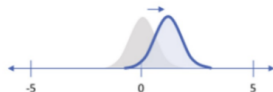


Image: Source

AE's: Variational Autoencoder (Details)

Penalizing reconstruction loss encourages the distribution to describe the input



Our distribution deviates from the prior to describe some characteristic of the data

Without regularization, our network can “cheat” by learning narrow distributions



With a small enough variance, this distribution is effectively only representing a single value

Penalizing KL divergence
acts as a regularizing force

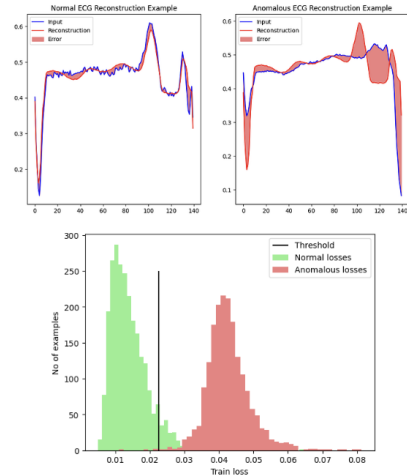
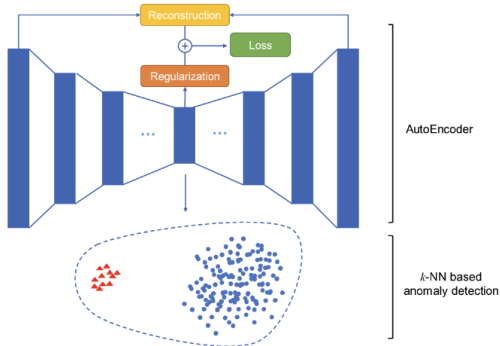
Attract distribution
to have zero mean



Ensure sufficient variance to yield a smooth latent space

The VAE objectives arranges data on a compact manifold (we can sample from) in a continuous smooth way.

AE other Applications: Anomaly Detection



AE other Applications: Image Colorization

